Important FastAPI Topics

1. Basics

- **What is FastAPI and why it's fast (ASGI, async/await, Starlette, Pydantic).**
- **Creating your first app with FastAPI().**
- **Running with uvicorn.**

---

2. Path & Query Parameters

- **Path parameters (/items/{id}).**
- **Query parameters (/items/?skip=10&limit=5).**
- **Optional and default values.**

---

3. Request Body

- **Sending JSON data in POST/PUT.**
- **Using Pydantic models for validation.**
- **Nested models.**

---

4. Response

- **Returning JSON, dicts, lists.**
- **Response models (Pydantic) to enforce structure.**
- **Status codes.**

---

5. Form & File Handling

- **Form for HTML form data.**
- **Uploading files with File and UploadFile.**

---

6. Error Handling

- **HTTPException.**
- **Custom error responses.**

- **Global exception handlers.**

---

7. Dependencies

- **Using Depends() for reusable logic.**
- **Dependencies for query params, authentication, pagination, etc.**
- **Sub-dependencies (dependencies inside dependencies).**

---

8. Security & Authentication

- **API keys (query/header).**
- **HTTP Basic Auth.**
- **OAuth2 with JWT tokens (real-world).**
- **Role-based access with dependencies.**

---

9. Middleware

- **What middleware is.**
- **Example: logging requests, CORS, custom headers.**

---

10. Background Tasks

- **Using BackgroundTasks to run jobs after response (e.g., sending email).**

---

11. Databases

- **Connecting with SQLAlchemy / Tortoise ORM.**
- **Async DB queries.**
- **Dependency-injected DB sessions.**

---

12. FastAPI Features

- **Automatic docs (/docs Swagger, /redoc).**
- **Response classes (HTMLResponse, FileResponse, RedirectResponse).**

- **Static files & templates (Jinja2).**

---

13. Testing

- **Using TestClient from fastapi.testclient.**
- **Writing unit tests for routes.**

---

14. Advanced

- **WebSockets.**
- **Event handlers (startup, shutdown).**
- **Caching with dependencies.**
- **Versioning APIs.**
- **Deploying with Docker / Uvicorn / Gunicorn.**

1. What is FastAPI?

- **FastAPI** is a modern, fast (high-performance) web framework for building APIs with **Python 3.7+**.
- Built on **Starlette** (for web handling) and **Pydantic** (for data validation).
- Used for **backend APIs**, ML model deployment, and microservices.

👉 Why FastAPI?

- Super fast
- Automatic **API Docs** (Swagger UI)
- Easy validation of data
- Asynchronous support (async/await)

---

2. Installation

Tell students to create a virtual environment first:

Create venv

```
python -m venv venv

# Activate

venv\Scripts\activate    # Windows

source venv/bin/activate # Mac/Linux

# Install FastAPI + Uvicorn (server)

pip install fastapi uvicorn
```

---

3. Hello World Example

File: main.py

```
from fastapi import FastAPI

# Create FastAPI app

app = FastAPI()

# Home route

@app.get("/")

def read_root():

    return {"message": "Hello, FastAPI!"}

# Another route

@app.get("/welcome/{name}")

def welcome_user(name: str):

    return {"message": f"Welcome, {name}!"}
```

Run the server:

```
uvicorn main:app --reload
```

- Open browser:
  👉 http://127.0.0.1:8000
  👉 Docs: http://127.0.0.1:8000/docs (Swagger)

---

1. What are Parameters in APIs?

Parameters are values we pass to an API to get **specific information**.
In FastAPI, we have two main types:

- **Path Parameters** → Part of the URL path.
- **Query Parameters** → Passed after ? in the URL.

---

2. Path Parameters

🔹 **Definition**

- A path parameter is written inside {} in the route.
- It is **mandatory** because it's part of the URL.

🔹 **Example**

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/students/{student_id}")

def get_student(student_id: int):

   return {"student_id": student_id}
```

🔹 **Test**

- URL: http://127.0.0.1:8000/students/101
- Response:

```
{"student_id": 101}
```

✅ Use Case: Get details of a **specific student, employee, product, etc.**

---

3. Query Parameters

🔹 **Definition**

- Query parameters come **after ? in the URL**.
- Multiple parameters are separated by &.
- They are usually **optional**.

🔹 **Example**

```
@app.get("/search")

def search_students(name: str, age: int = None):
```

return {"name": name, "age": age}

🔹 **Test**

- URL: http://127.0.0.1:8000/search?name=Sneha&age=23
- Response:

{"name": "Sneha", "age": 23}

- URL: http://127.0.0.1:8000/search?name=Rajan
- Response:

{"name": "Rajan", "age": null}

✅ Use Case: Searching or filtering.

=>What is Request Body in FastAPI?

1️⃣ **Simple Definition**

- **Request Body** = The data you send **inside the request** (not in the URL).
- Usually in **JSON format**.
- Used when you want to send **complex data** like employee info, student details, login credentials, etc.

2️⃣ **Example without Request Body (using query/path)**

@app.get("/student")

def get_student(name: str, age: int):

  return {"name": name, "age": age}

➡️ URL must include values:
http://127.0.0.1:8000/student?name=Sneha&age=23

This works for **small/simple data**, but not for complex objects.

3️⃣ **Request Body with Pydantic (Best Practice)**

👉 Define a model using **Pydantic** (BaseModel):

from fastapi import FastAPI

```
from pydantic import BaseModel

app = FastAPI()

# Define model

class Student(BaseModel):

    id: int

    name: str

    age: int

    grade: str

# Use Request Body

@app.post("/students")

def create_student(student: Student):

    return {"message": "Student created", "student": student}
```

=>**What is Form Data?**

- **Form data** is used when data is submitted from an **HTML <form>**.
- Instead of JSON in the body, data comes as **key-value pairs**.
- Mostly used for:
    - Login forms (username + password)
    - File upload with text fields
    - Contact forms

1️⃣ **Example: Login with Form Data**

```
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/login")

def login(username: str = Form(...), password: str = Form(...)):

    return {"username": username, "password": password}
```

Explanation

Form(...) tells FastAPI that these values will come from **form fields**.

Not JSON.

Example: Signup with Form Data

```
@app.post("/signup")
def signup(
    name: str = Form(...),
    email: str = Form(...),
    password: str = Form(...)
):
    return {"message": "User signed up", "name": name, "email": email}
```

=>File Uploads in FastAPI

FastAPI provides two classes for this:

- **File** → for receiving files.
- **UploadFile** → a more advanced way (gives filename, content type, etc.).

1️⃣ Basic Example: Upload a File

```
from fastapi import FastAPI, File, UploadFile

app = FastAPI()

@app.post("/uploadfile/")
async def upload_file(file: UploadFile = File(...)):
    return {
        "filename": file.filename,
        "content_type": file.content_type
    }
```

ERROR HANDLING

1. What is an HTTP Exception?

In FastAPI, when something goes wrong (like resource not found, invalid input, unauthorized access), you should return an appropriate HTTP error response.

Instead of just returning plain text, FastAPI provides a built-in way: HTTPException.

2. Importing HTTPException

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
```

3. Raising HTTPException

Example: A fake database of items.

```
items = {"1": "Apple", "2": "Banana"}


@app.get("/items/{item_id}")
def get_item(item_id: str):
    if item_id not in items:
        # Raise 404 error if item is not found
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item_id": item_id, "name": items[item_id]}
```

If you request /items/1, you get :

```
{"item_id": "1", "name": "Apple"}
```

If you request /items/10, you get :

```
{
  "detail": "Item not found"
}
```

4. Parameters of HTTPException

```
raise HTTPException(
    status_code=403,   # HTTP status (e.g. 403 Forbidden)
    detail="You don't have access",  # message
    headers={"X-Error": "Custom error"}  # optional custom headers
)
```

5. Common Status Codes

404 → Not Found

400 → Bad Request

401 → Unauthorized

403 → Forbidden

500 → Internal Server Error

Example: Greeting with a Dependency

```
from fastapi import FastAPI, Depends

app = FastAPI()

# Step 1: Create a dependency function
def get_name():
    return "Sneha"

# Step 2: Use Depends in a route
@app.get("/hello/")
def say_hello(name: str = Depends(get_name)):
    return {"message": f"Hello {name}!"}
```

Background Tasks in FastAPI

Why use Background Tasks?

Sometimes, you don't want to keep the client waiting while you finish heavy work.

Example: A user signs up → API should return "Account created" immediately, while in the background you send a welcome email.

FastAPI has a built-in BackgroundTasks class for this.

Example 1: Simple Background Task

```
from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

# Task function

def write_log(message: str):

    with open("log.txt", "a") as f:

        f.write(message + "\n")

@app.post("/submit/")

def submit(background_tasks: BackgroundTasks):

    background_tasks.add_task(write_log, "User submitted data")

    return {"message": "Data received! Log will be written in background."}
```

How it works:

User calls /submit/.

FastAPI returns response immediately → {"message": "Data received!"}.

In the background, write_log("User submitted data") is executed → message is written to log.txt.

Example 2: Sending Email (Simulation)

```
from fastapi import FastAPI, BackgroundTasks

import time

app = FastAPI()

def send_email(email: str, message: str):

    time.sleep(5)  # Simulate delay

    with open("emails.txt", "a") as f:

        f.write(f"To: {email} | Message: {message}\n")

@app.post("/register/")

def register(email: str, background_tasks: BackgroundTasks):

    background_tasks.add_task(send_email, email, "Welcome to FastAPI!")
```

return {"message": f"User registered with {email}. Email will be sent shortly."}

API responds instantly.

Email (simulated with emails.txt) is sent in the background.

Example 3: Multiple Tasks

```
@app.post("/process/")
def process(background_tasks: BackgroundTasks):
    background_tasks.add_task(write_log, "Process started")
    background_tasks.add_task(send_email, "user@example.com", "Processing done")
    return {"message": "Tasks scheduled!"}
```

Middleware in FastAPI

What is Middleware?

Middleware is code that runs before and after every request.

It sits between the client request and the API response.

Useful for:

Logging

Security checks

Adding headers

Measuring execution time

Handling CORS (Cross-Origin Resource Sharing)

Example 1: Logging Middleware

```
import time
from fastapi import FastAPI, Request
app = FastAPI()
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start_time = time.time()
```

```
    # Process the request
    response = await call_next(request)
    # Log details after response
    duration = time.time() - start_time
    print(f"{request.method} {request.url} completed in {duration:.2f}s")
    return response
@app.get("/")
async def home():
    return {"message": "Hello Middleware!"}
```

What happens

Before hitting /, middleware records start time.

After response, it prints execution time.

Example 2: Add Custom Header

```
@app.middleware("http")
async def add_custom_header(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Custom-Header"] = "FastAPI Middleware"
    return response

@app.get("/items/")
async def get_items():
    return {"items": ["apple", "banana", "mango"]}
```

Every response will now include a header:

X-Custom-Header: FastAPI Middleware

Example 3: Block Unauthorized Requests

```
@app.middleware("http")
```

```
async def check_api_key(request: Request, call_next):
    api_key = request.headers.get("x-api-key")
    if api_key != "secret123":
        return JSONResponse(status_code=401, content={"detail": "Unauthorized"})
    return await call_next(request)

@app.get("/secure/")
async def secure_data():
    return {"message": "You passed middleware security!"}
```

If request doesn't include header x-api-key: secret123, it returns 401 Unauthorized.

Example 4: CORS Middleware (Built-in)

FastAPI provides ready-to-use CORS middleware.

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],   # allow all origins (use specific domains in real apps)
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

This allows frontend apps (like React, Angular) to call your API without CORS errors.

WebSockets in FastAPI

What are WebSockets?

Unlike HTTP (request → response), WebSockets provide a persistent, two-way connection between client and server.

Useful for:

Chat apps

Real-time notifications

Live dashboards (stock prices, IoT data)

Collaborative apps (Google Docs style)

Example 1: Basic WebSocket Echo

```
from fastapi import FastAPI, WebSocket
app = FastAPI()
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    # Accept connection
    await websocket.accept()
    while True:
        # Receive message from client
        data = await websocket.receive_text()
        # Send it back (echo)
        await websocket.send_text(f"Message received: {data}")
```

How it works:

Client connects → /ws.

Server accepts.

Client sends "Hello".

Server replies "Message received: Hello".

Example 2: Chat Between Multiple Clients

```
from fastapi import FastAPI, WebSocket
from typing import List
app = FastAPI()
# Store active connections
class ConnectionManager:
    def __init__(self):
```

```
self.active_connections: List[WebSocket] = []
    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)
    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)
    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)
manager = ConnectionManager()
@app.websocket("/chat")
async def chat(websocket: WebSocket):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            await manager.broadcast(f"User says: {data}")
    except:
        manager.disconnect(websocket)
```

Now, multiple clients can join /chat and talk in real time.

Example 3: WebSocket with Query Params

```
@app.websocket("/ws/{client_id}")
async def websocket_with_id(websocket: WebSocket, client_id: int):
    await websocket.accept()
    await websocket.send_text(f"Hello Client {client_id}")
```

Each client gets a custom welcome message.

Example 4: Frontend (HTML + JS Client)

Save this as index.html and open in browser:

```
<!DOCTYPE html>
<html>
  <body>
    <h2>WebSocket Chat</h2>
    <input id="msg" placeholder="Type a message..." />
    <button onclick="sendMessage()">Send</button>
    <ul id="messages"></ul>
    <script>
      const ws = new WebSocket("ws://localhost:8000/chat");
      ws.onmessage = function(event) {
        const li = document.createElement("li");
        li.innerText = event.data;
        document.getElementById("messages").appendChild(li);
      };
      function sendMessage() {
        const msg = document.getElementById("msg").value;
        ws.send(msg);
      }
    </script>
  </body>
</html>
```

Connect fastapi with Python (2 methods)

FirsT Method:Database.py

```
import mysql.connector
from mysql.connector import Error
def create_table():
    try:
```

```
        connection = mysql.connector.connect(
            host="localhost",
            user="root",
            password="your password",
            database="db name"
        )
        cursor = connection.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS students (
                id INT AUTO_INCREMENT PRIMARY KEY,
                name VARCHAR(50),
                age INT
            )
        """)
        connection.commit()
        cursor.close()
        connection.close()
        print(" Table created successfully")
    except Error as e:
        print(f" MySQL Error: {e}")
```

main.py

```
 from fastapi import FastAPI
from app.database import create_table

app = FastAPI()

# Run DB setup on startup
@app.on_event("startup")
def startup_event():
    create_table()

@app.get("/")
def home():
    return {"message": "Hello FastAPI with DB!"}
```

Second method :Main.py

```
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
import models
from database import engine, Base, get_db
```

```
# Create tables
Base.metadata.create_all(bind=engine)
app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello World"}

@app.post("/students/")
def add_student(name: str, age: int, db: Session = Depends(get_db)):
    student = models.Student(name=name, age=age)
    db.add(student)
    db.commit()
    db.refresh(student)
    return student

@app.get("/students/")
def get_students(db: Session = Depends(get_db)):
    return db.query(models.Student).all()
```

Models.py

```
from sqlalchemy import Column, Integer, String
from database import Base

class Student(Base):
    __tablename__ = "students"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(50))
    age = Column(Integer)
```

Database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "mysql+pymysql://root:password@localhost/db_name"

engine = create_engine(DATABASE_URL, echo=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
```

```
db = SessionLocal()
try:
    yield db
finally:
    db.close()
```